BREAKOUT GAME DEVELOPMENT USING PYTHON

AIM:

To design and develop an interactive 2D Breakout Game using Python Tkinter that allows players to control a paddle using the left and right arrow keys, launch a ball using the spacebar, destroy bricks, and increase their score. The game includes ball collision detection, lives tracking, win and game-over detection, and restart and exit options.

PROCEDURE:

1. Start the program and import the required Python libraries: Tkinter and random.
2. Create the main game window using Tkinter and set its title to "BREAKOUT GAME".
3. Design the graphical interface using labels, buttons, frames, and a Canvas widget.
4. Create five rows of coloured bricks, a paddle, and a ball inside the game area.
5. Initialize the score to 0 and the player's lives to 3.
6. Implement keyboard controls using the left and right arrow keys to move the paddle.
7. Use the spacebar to launch the ball from the paddle.
8. Update the ball's position continuously using Tkinter's after() method to create animation.
9. Implement collision detection so the ball bounces off the walls, paddle, and bricks.
10. Increase the score by 10 points whenever a brick is destroyed and update the score display.

11. Reduce the player's lives when the ball falls below the paddle. Display “GAME OVER” when all lives are lost and “YOU WIN!” when all bricks are destroyed.
12. Provide Restart Game and Exit buttons, execute the program, and verify the game controls, scoring, collision detection, and restart functionality.

PROGRAM CODE :

```
import tkinter as tk
import random
# Game Settings
WIDTH = 600
HEIGHT = 450
PADDLE_W = 100
PADDLE_H = 12
PADDLE_SPEED = 20
PADDLE_Y = HEIGHT - 30
BALL_SIZE = 12
BALL_SPEED = 5
BRICK_ROWS = 5
BRICK_COLS = 10
BRICK_W = 54
BRICK_H = 18
BRICK_GAP = 6
TOP_OFFSET = 60
DELAY = 16  # about 60 frames per second
COLORS = ["#EF4444", "#F97316", "#FACC15", "#22C55E", "#38BDF8"]
class BreakoutGame:
    def __init__(self, root):
        self.root = root
```

```
 self.root.title("BREAKOUT GAME")
 self.root.resizable(False, False)
 self.root.configure(bg="#172554")

 tk.Label(
     root,
     text="BREAKOUT GAME",
     font=("Arial Black", 22),
     bg="#172554",
     fg="white"
 ).pack(pady=10)

 info_frame = tk.Frame(root, bg="#172554")
 info_frame.pack()
     self.score_label = tk.Label(
     info_frame,
     text="SCORE: 0",
     font=("Arial", 14, "bold"),
     bg="#172554",
     fg="#FACC15"
 )
 self.score_label.pack(side="left", padx=30)
self.lives_label = tk.Label(
     info_frame,
     text="LIVES: 3",
     font=("Arial", 14, "bold"),
     bg="#172554",
     fg="#86EFAC"
 )
```

```
self.lives_label.pack(side="left", padx=30)
self.canvas = tk.Canvas(
    root,
    width=WIDTH,
    height=HEIGHT,
    bg="#0F172A",
    highlightthickness=2,
    highlightbackground="#38BDF8"
)
self.canvas.pack(padx=20, pady=10)
button_frame = tk.Frame(root, bg="#172554")
button_frame.pack(pady=10)
tk.Button(
    button_frame,
    text="RESTART GAME",
    font=("Arial", 12, "bold"),
    bg="#22C55E",
    fg="black",
    command=self.restart
).pack(side="left", padx=10)
  tk.Button(
    button_frame,
    text="EXIT",
    font=("Arial", 12, "bold"),
    bg="#EF4444",
    fg="white",
    command=root.destroy
).pack(side="left", padx=10)
  root.bind("<KeyPress>", self.on_key_press)
```

```
        self.timer = None
        self.restart()

    def restart(self):
        if self.timer is not None:
            self.root.after_cancel(self.timer)
            self.timer = None

        self.score = 0
        self.lives = 3
        self.running = True
        self.launched = False
        self.paddle_x = (WIDTH - PADDLE_W) // 2

        self.score_label.config(text="SCORE: 0")
        self.lives_label.config(text="LIVES: 3")

        self.create_bricks()
        self.reset_ball()
        self.draw_game()
        self.update_game()

    def create_bricks(self):
        self.bricks = []
        total_width = BRICK_COLS * BRICK_W + (BRICK_COLS - 1) * BRICK_GAP
        left = (WIDTH - total_width) // 2
        for row in range(BRICK_ROWS):
            color = COLORS[row % len(COLORS)]
            y = TOP_OFFSET + row * (BRICK_H + BRICK_GAP)
```

```
        for col in range(BRICK_COLS):
            x = left + col * (BRICK_W + BRICK_GAP)
            self.bricks.append((x, y, color))

def reset_ball(self):
    self.launched = False
    self.ball_x = self.paddle_x + PADDLE_W // 2 - BALL_SIZE // 2
    self.ball_y = PADDLE_Y - BALL_SIZE
    self.dx = random.choice([-4, 4])
    self.dy = -BALL_SPEED

def on_key_press(self, event):
    if not self.running:
        if event.keysym in ("r", "R"):
            self.restart()
        return

    if event.keysym == "Left":
        self.paddle_x = max(0, self.paddle_x - PADDLE_SPEED)
    elif event.keysym == "Right":
        self.paddle_x = min(WIDTH - PADDLE_W, self.paddle_x + PADDLE_SPEED)
    elif event.keysym == "space":
        self.launched = True
    elif event.keysym in ("r", "R"):
        self.restart()

def update_game(self):
    if not self.running:
        return
```

```
        if self.launched:
            self.ball_x += self.dx
            self.ball_y += self.dy

            # Wall collisions
            if self.ball_x <= 0 or self.ball_x + BALL_SIZE >= WIDTH:
                self.dx = -self.dx
                self.ball_x = max(0, min(self.ball_x, WIDTH - BALL_SIZE))
            if self.ball_y <= 0:
                self.dy = -self.dy
                self.ball_y = 0

            # Paddle collision
            if (
                self.dy > 0 and
                PADDLE_Y <= self.ball_y + BALL_SIZE <= PADDLE_Y + PADDLE_H +
BALL_SPEED and
                self.paddle_x <= self.ball_x + BALL_SIZE and
                self.ball_x <= self.paddle_x + PADDLE_W
            ):
                self.dy = -abs(self.dy)
                # The hit position on the paddle sets the bounce angle
                offset = (self.ball_x + BALL_SIZE / 2) - (self.paddle_x + PADDLE_W / 2)
                self.dx = offset / (PADDLE_W / 2) * 6
                self.ball_y = PADDLE_Y - BALL_SIZE

            # Brick collisions
            for brick in self.bricks:
                bx, by, _ = brick
```

```
            if (
                self.ball_x < bx + BRICK_W and
                self.ball_x + BALL_SIZE > bx and
                self.ball_y < by + BRICK_H and
                self.ball_y + BALL_SIZE > by
            ):
                self.bricks.remove(brick)
                self.dy = -self.dy
                self.score += 10
                self.score_label.config(text=f"SCORE: {self.score}")
                break

        # Ball fell below the paddle
        if self.ball_y > HEIGHT:
            self.lives -= 1
            self.lives_label.config(text=f"LIVES: {self.lives}")
            if self.lives == 0:
                self.game_over("GAME OVER")
                return
            self.reset_ball()

        # All bricks destroyed
        if not self.bricks:
            self.game_over("YOU WIN!")
            return
    else:
        # Ball stays on the paddle until launched
        self.ball_x = self.paddle_x + PADDLE_W // 2 - BALL_SIZE // 2
```

```
    self.draw_game()
    self.timer = self.root.after(DELAY, self.update_game)

def draw_game(self):
    self.canvas.delete("all")

    # Draw bricks
    for x, y, color in self.bricks:
        self.canvas.create_rectangle(
            x, y, x + BRICK_W, y + BRICK_H,
            fill=color,
            outline=""
        )

    # Draw paddle
    self.canvas.create_rectangle(
        self.paddle_x, PADDLE_Y,
        self.paddle_x + PADDLE_W, PADDLE_Y + PADDLE_H,
        fill="#86EFAC",
        outline="#14532D"
    )

    # Draw ball
    self.canvas.create_oval(
        self.ball_x, self.ball_y,
        self.ball_x + BALL_SIZE, self.ball_y + BALL_SIZE,
        fill="white",
        outline=""
    )
```

```
    if not self.launched and self.running:
        self.canvas.create_text(
            WIDTH // 2, HEIGHT - 80,
            text="Press SPACE to launch",
            font=("Arial", 12),
            fill="#94A3B8"
        )

def game_over(self, message):
    self.running = False
    self.timer = None
    colour = "#EF4444" if message == "GAME OVER" else "#FACC15"

    self.canvas.create_rectangle(
        150, 160, 450, 290,
        fill="#172554",
        outline="#38BDF8",
        width=2
    )
    self.canvas.create_text(
        WIDTH // 2, 200,
        text=message,
        font=("Arial Black", 26),
        fill=colour
    )
    self.canvas.create_text(
        WIDTH // 2, 250,
        text=f"Final Score: {self.score}   (press R to restart)",
```

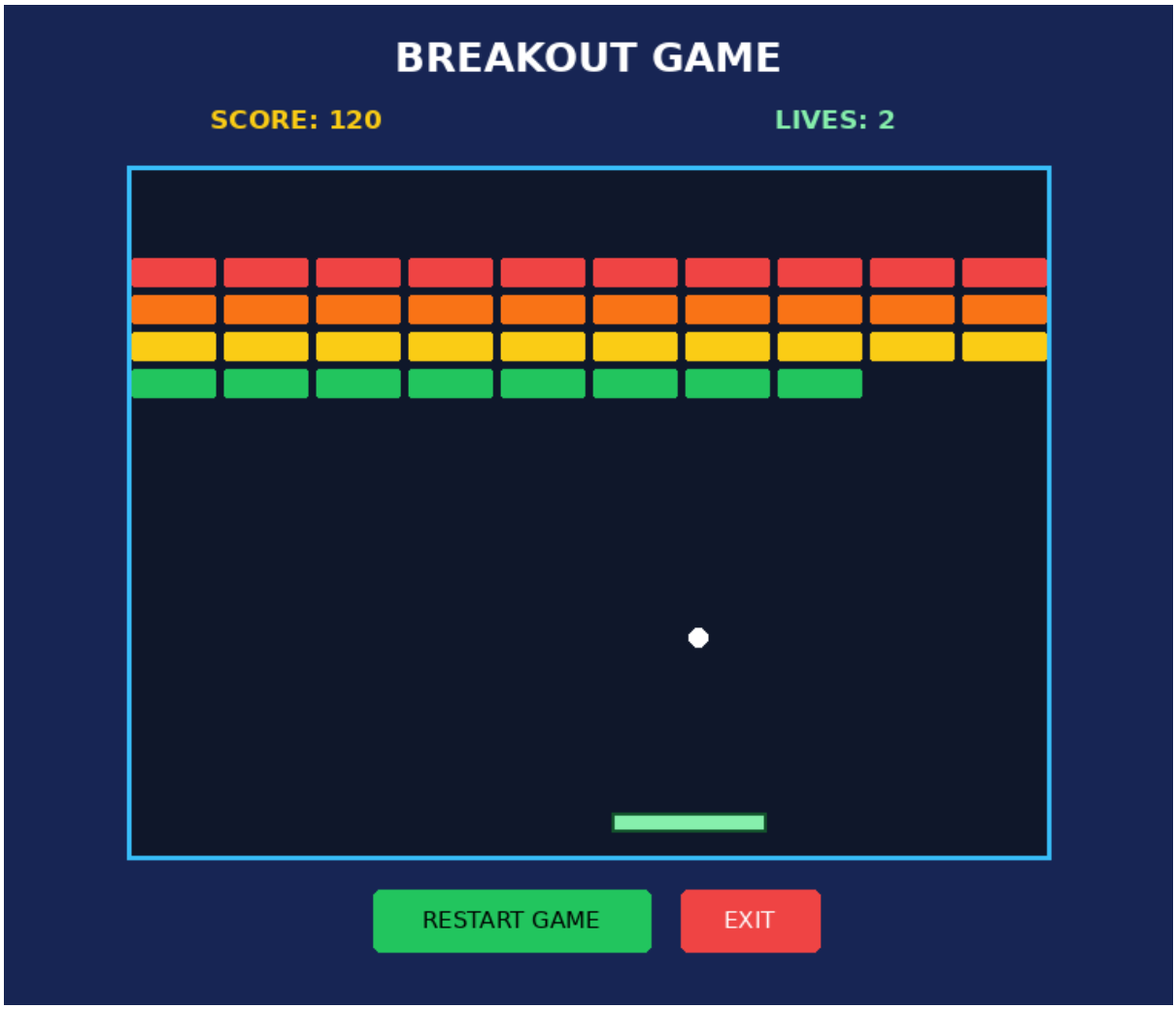
```
            font=("Arial", 14),
            fill="white"
        )
# Main Application
root = tk.Tk()
game = BreakoutGame(root)
root.mainloop()
```

OUTPUT:

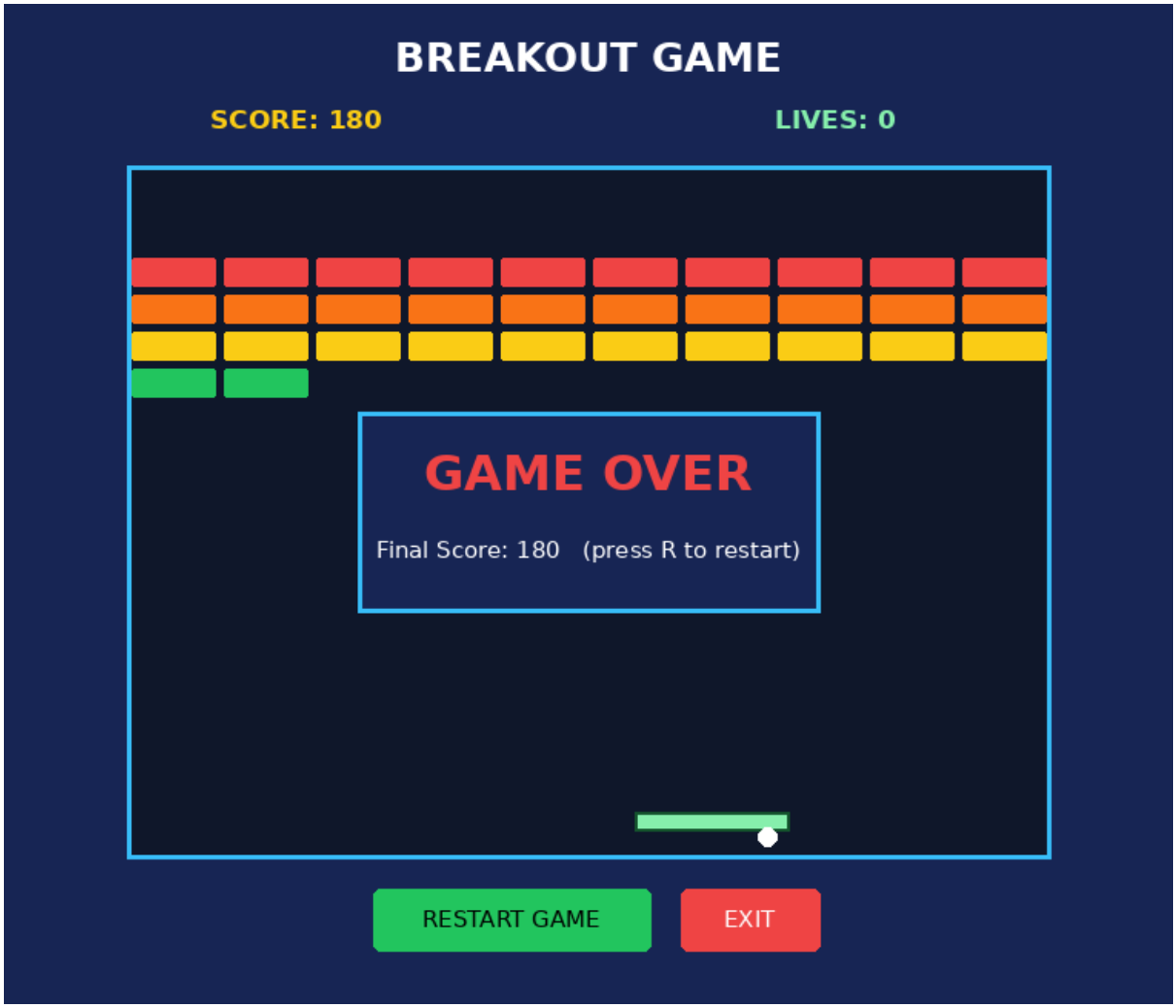
#INITIAL GAME



#GAME PLAY



#GAME OVER



RESULT:

Thus, an interactive 2D Breakout Game was developed using Python Tkinter with a graphical game interface, keyboard-based paddle movement, ball launching and bouncing, brick destruction, score calculation, lives tracking, win and game-over handling, restart functionality, and exit controls. The program demonstrates event-driven programming, GUI design, animation, collision detection, conditional statements, and basic game development concepts.